

자전거

APIO 시에서 APIOBike라는 새로운 자전거 공유 서비스가 시작되었다. 도시 여러 곳에 자전거 주차장이 설치되어서, 이 중 어느 곳에서도 자전거를 빌리거나 반납할 수 있다. 자전거 공유는 편하고 싸기 때문에 APIO 시의 주요 교통수단이 되었다. 그렇지만 APIOBike에서도 다른 자전거 공유 서비스에도 흔한 문제가 생겼다. 주차장마다 빌리고 반납하는 비율이 다른 것이다. 이 결과로 어떤 주차장에는 자전거가 거의 없어서 사람들이 빌릴 수 없어지고, 다른 주차장에는 사람들이 필요로 하지 않은 자전거가 쌓이게 되었다.

이 문제를 해결하기 위해서, 트럭으로 자전거를 실어서 주차장마다 자전거 수를 균형을 맞추기로 했다.

APIO 시에는 주차장이 N 개 있는데, $0, 1, \dots, N - 1$ 로 번호가 매겨져 있다. 관찰을 통해서 매일 저녁 주차장 i 에는 항상 자전거 $A[i]$ 대가 있고 밤에는 사람들이 자전거를 빌리거나 반납하지 않는 것을 알았다. APIOBike는 매일 아침 주차장 i 에 정확히 자전거 $B[i]$ 대가 있게 하려고 한다.

N 개의 주차장 사이에는 트럭이 전용으로 쓸 수 있는 $N - 1$ 개의 도로가 있다. 각 도로는 서로 다른 두 주차장을 연결하며, 트럭은 이 도로들로만 다닐 수 있다. 모든 도로의 길이는 1이다. 주차장의 네트워크는 연결되어 있어서, 트럭은 어떤 두 주차장 사이로도 이동할 수 있다. 매일 밤, APIOBike는 트럭을 보내서 주차장 i 에 정확히 자전거 $B[i]$ 대가 있게 하려고 한다. 트럭은 임의의 주차장에서 출발해도 되고, 임의의 주차장에서 종료해도 된다.

처음 출발할 때 트럭은 비어있다. 트럭이 주차장에 정차했을 때, 운전사는 이 주차장에 있는 자전거를 트럭에 싣거나, 트럭에 있는 자전거를 주차장에 내려놓을 수 있고 싣거나 내려놓는 자전거의 수는 제한이 없다. 트럭에 싣을 수 있는 자전거의 수와 주차장에 세워놓을 수 있는 자전거의 수는 제한이 없지만, 어느 순간에도 절대로 0보다 작을 수는 없다. APIOBike는 총 이동거리를 최소화해서 자전거 수를 균형을 맞추고 싶어하고, 어떻게 하면 되는지 알고 싶어한다.

Implementation Details

다음 함수를 구현해야 한다.

```
std::pair<std::vector<int>, std::vector<long long>>
find_rebalancing_strategy(int N,
                           std::vector<int> A,
                           std::vector<int> B,
                           std::vector<int> U,
                           std::vector<int> V)
```

- A : 길이 N 인 배열로 $A[i]$ 는 매일 저녁주차장 i 에 있는 자전거 수이다.

- B : 길이 N 인 배열로 $B[i]$ 는 매일 아침 주차장 i 에 있게 하려는 자전거 수이다.
- U, V : 길이 $N - 1$ 인 배열로 도로 네트워크를 표현한다. 각 $0 \leq i < N - 1$ 에 대해서 i 번째 도로는 주차장 $U[i]$ 과 주차장 $V[i]$ 를 연결한다.
- 이 함수는 각각의 테스트 케이스에 대해 **최대 150 000 번** 호출될 수 있다.

이 함수는 길이가 각각 $k + 1$ 인 두 배열 (X, Y) 을 리턴하는데, 자전거 수 균형을 맞추는 방법을 표현한다.

- X : 차례대로 방문해야 할 주차장 번호
- Y : 각 주차장에서 할 일들. 각 $0 \leq j \leq k$ 에 대해서
 - 만약 $Y[j] \geq 0$ 이면, 운전사는 트럭에서 $Y[j]$ 대의 자전거를 주차장 $X[j]$ 에 내려놓는다. 즉, 주차장 $X[j]$ 의 자전거 수는 $Y[j]$ 만큼 늘어난다.
 - 만약 $Y[j] < 0$ 이면, 운전사는 주차장 $X[j]$ 에서 $-Y[j]$ 대의 자전거를 트럭에 싣는다. 즉, 주차장 $X[j]$ 의 자전거 수는 $-Y[j]$ 만큼 줄어든다.

거리 k 인 **타당한** 자전거 수 균형을 맞추는 방법 (X, Y) 은 다음 조건을 만족해야 한다.

- 각 $0 \leq j \leq k$ 에 대해서, $0 \leq X[j] < N$.
- 각 $0 \leq j < k$ 에 대해서, 주차장 $X[j]$ 와 주차장 $X[j + 1]$ 는 도로 하나로 직접 연결되어야 한다.
- 각 $0 \leq j \leq k$ 에 대해서, $\sum_{t=0}^j Y[t] \leq 0$.
- 각 주차장 $0 \leq i < N$ 와 각 단계 $0 \leq j \leq k$ 에 대해서, $S_{i,j}$ 가 $0 \leq t \leq j$ 이고 $X[t] = i$ 인 모든 단계 t 에 대해서 $Y[t]$ 의 총합이라고 하자. 그러면,
 - 주차장 i 에 있는 자전거 수는 절대로 0보다 작을 수 없다: $A[i] + S_{i,j} \geq 0$.
 - 종료시 주차장 i 에는 정확히 자전거 $B[i]$ 대가 있어야 한다: $A[i] + S_{i,k} = B[i]$.

Constraints

- $2 \leq N \leq 300\,000$
- 각 테스트 케이스에서, `find_rebalancing_strategy` 호출 전체의 N 값의 합은 300 000을 넘지 않는다.
- $0 \leq i < N$ 인 각 i 에 대해서 $0 \leq U[i], V[i] < N$ 이고 $U[i] \neq V[i]$.
- $0 \leq i < N$ 인 각 i 에 대해서 $0 \leq A[i], B[i] \leq 10^9$.
- 어떤 주차장 쌍이 주어져도 이 둘은 연결되어 있다.
- $\sum_{i=0}^{N-1} A[i] = \sum_{i=0}^{N-1} B[i]$.
- $0 \leq i < N$ 이고 $A[i] \neq B[i]$ 인 i 가 최소한 하나 있다.

Subtasks and Scoring

T 가 `find_rebalancing_strategy` 호출 횟수이고, $\sum N$ 가 전체 `find_rebalancing_strategy` 호출에 대한 N 값들의 합이라고 하자.

서브태스크	점수	추가적인 제약 조건
1	4	주차장과 도로는 아래에 설명하는 속성 P를 만족한다. $0 \leq i < N$ 이고 $A[i] > 0$ 인 i 는 정확히 하나 있다.
2	11	$T \leq 10$. $N \leq 7$. 주차장과 도로는 속성 P를 만족한다.
3	16	주차장과 도로는 속성 P를 만족한다.
4	9	$0 \leq i < N$ 이고 $A[i] > 0$ 인 i 는 정확히 하나 있다.
5	24	$\sum N \leq 500$.
6	15	$\sum N \leq 5000$.
7	21	추가적인 제약 조건이 없다.

속성 P: 각 $0 \leq i < N - 1$ 에 대해서, $U[i] = i$ 이고 $V[i] = i + 1$ 이다. 각 $0 \leq i < N$ 에 대해서, $A[i] \neq B[i]$ 이다.

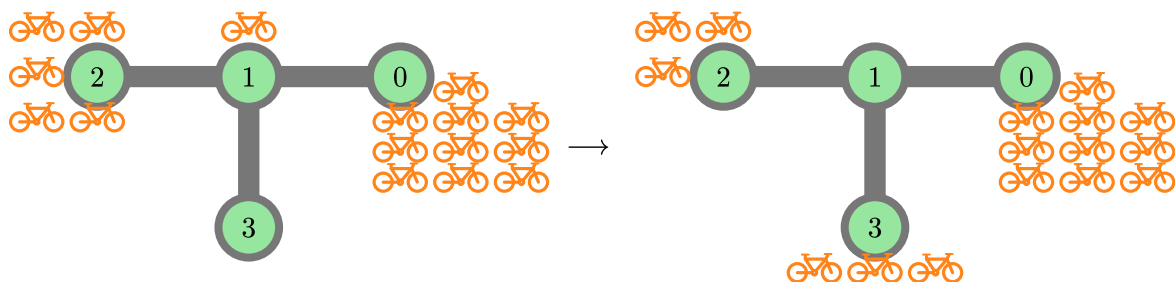
만약 리턴한 배열 X, Y 의 길이가 정확히 $k^* + 1$ 이지만 (k^* 는 최소 가능한 이동 거리) (X, Y) 가 타당한 자전거 수 균형을 맞추는 방법이 아니라면, 점수의 50%를 받는다.

Example

Example 1

다음 호출을 생각해보자.

```
find_rebalancing_strategy(4,
                        [10, 1, 5, 0],
                        [10, 0, 3, 3],
                        [0, 1, 1],
                        [1, 2, 3])
```



APIO 시에는 $N = 4$ 개의 주차장이 있다. 처음에 주차장에 있는 자전거 수는 $A = [10, 1, 5, 0]$ 인데, 목표는 자전거 수를 $B = [10, 0, 3, 3]$ 로 하는 것이다.

트럭이 주차장 2에서 출발해서 다음과 같은 일을 한다고 하자.

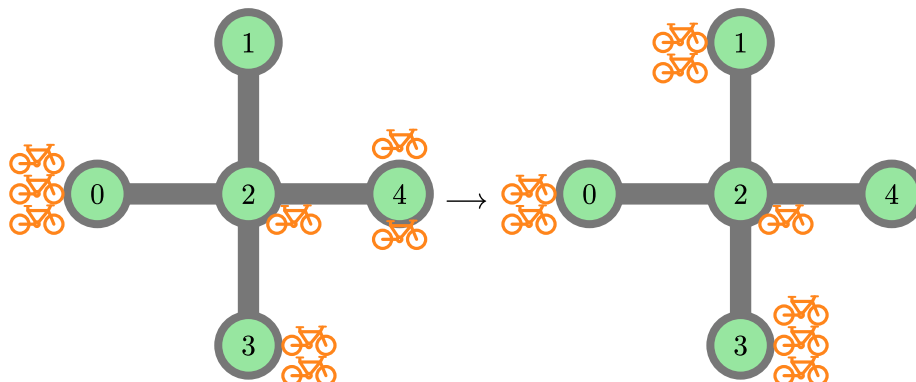
- 주차장 2에서 2대의 자전거를 트럭에 싣는다. 주차장 2에는 이제 자전거가 3대 있고, 트럭에는 자전거가 2대 있다.
- 주차장 1로 이동해서 1대의 자전거를 트럭에 싣는다. 주차장 1에는 이제 자전거가 0대 있고, 트럭에는 자전거가 3대 있다.
- 주차장 3로 이동해서 3대의 자전거를 내려놓는다. 주차장 3에는 이제 자전거가 3대 있고, 트럭에는 자전거가 0대 있다.

전체 이동 거리는 2이다. 이 방법을 표현하는 배열은 $X = [2, 1, 3]$, $Y = [-2, -1, 3]$ 이다. 이 방법이 최소 이동 거리로 문제를 푼다는 것을 증명할 수 있다. 따라서, 리턴 값은 $([2, 1, 3], [-2, -1, 3])$ 이어야 한다.

Example 2

다음 호출을 생각해보자.

```
find_rebalancing_strategy(5,
                        [3, 0, 1, 2, 2],
                        [2, 2, 1, 3, 0],
                        [2, 2, 2, 2],
                        [0, 4, 3, 1])
```



APIO 시에는 $N = 5$ 개의 주차장이 있다. 처음에 주차장에 있는 자전거 수는 $A = [3, 0, 1, 2, 2]$ 인데, 목표는 자전거 수를 $B = [2, 2, 1, 3, 0]$ 로 하는 것이다.

트럭이 주차장 0에서 출발해서 다음과 같은 일을 한다고 하자.

- 주차장 0에서 1대의 자전거를 트럭에 싣는다.
- 주차장 2로 이동해서 1대의 자전거를 트럭에 싣는다.
- 주차장 1로 이동해서 2대의 자전거를 내려놓는다.

- 주차장 2로 이동한다.
- 주차장 4로 이동해서 2대의 자전거를 트럭에 싣는다.
- 주차장 2로 이동해서 1대의 자전거를 내려놓는다.
- 주차장 3로 이동해서 1대의 자전거를 내려놓는다.

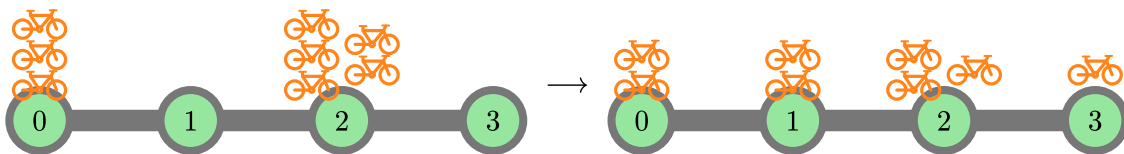
전체 이동 거리는 6이다. 이 방법을 표현하는 배열은 $X = [0, 2, 1, 2, 4, 2, 3]$, $Y = [-1, -1, 2, 0, -2, 1, 1]$ 이다. 이 방법이 최소 이동 거리로 문제를 푼다는 것을 증명할 수 있다. 따라서, 리턴 값은 $([0, 2, 1, 2, 4, 2, 3], [-1, -1, 2, 0, -2, 1, 1])$ 이어야 한다.

거리 6인 다른 타당한 전략도 있는데, 예를 들면 $X = [0, 2, 3, 2, 4, 2, 1]$, $Y = [-1, 0, 1, 0, -2, 0, 2]$ 이고 이 또한 정답으로 인정된다. 길이 $7 = 6 + 1$ 이지만 타당하지 않은 전략, 예를 들어 $X = Y = [0, 0, 0, 0, 0, 0, 0]$ 을 리턴하면 점수의 50%를 받는다.

Example 3

다음 호출을 생각해보자.

```
find_rebalancing_strategy(4,
                        [3, 0, 5, 0],
                        [2, 2, 3, 1],
                        [0, 1, 2],
                        [1, 2, 3])
```



최적의 해는 $X = [2, 1, 0, 1, 2, 3]$, $Y = [-1, 1, -1, 1, -1, 1]$ 이다. 리턴 값은 $([2, 1, 0, 1, 2, 3], [-1, 1, -1, 1, -1, 1])$. 다른 타당한 전략, 예를 들어 $X = [2, 1, 0, 1, 2, 3]$, $Y = [-2, 2, -1, 0, 0, 1]$ 도 정답으로 인정된다.

이 예제는 서브태스크 2, 3의 제약조건을 만족한다.

Sample Grader

입력의 첫 줄에는 시나리오의 수를 나타내는 하나의 정수 T 가 주어진다. T 개의 시나리오에 대한 설명이 아래와 같은 양식으로 주어진다.

입력 양식:

```
N
A[0] A[1] ... A[N-1]
B[0] B[1] ... B[N-1]
U[0] V[0]
U[1] V[1]
...
U[N-2] V[N-2]
```

출력 양식:

```
k
X[0] X[1] ... X[k]
Y[0] Y[1] ... Y[k]
```